Final Review Guide — Wine Quality Prediction

The whole project, explained from zero: the ideas, the story, the code, and the questions

Surianandhan S · 127018060 · CSE425 Machine Learning Essentials

How to use this guide tonight

Part 1 — Machine learning from zero

1.1 What is machine learning?
1.2 The words for data
1.3 Classification or regression?
1.4 Training and testing — the exam rule
1.5 Cross-validation — giving every wine a turn as the exam
1.6 Data leakage — peeking at the exam answers
1.7 Scaling (min–max normalisation)
1.8 How we score a model
1.9 Class imbalance — why the rare grades get ignored
1.10 The models, one by one (with everyday pictures)
1.11 Ensembles — the wisdom of the crowd
1.12 Stacking — let a model learn how to combine
1.13 Feature importance — which measurements matter?
1.14 A little statistics

Part 2 — The project story

2.1 The problem
2.2 The dataset
2.3 The base paper
2.4 Review 1 — the reproduction
2.5 What the guide asked after Review 1
2.6 Review 2 — change 1: class balancing
2.7 Review 2 — change 2: improving the result
2.8 Review 2 — change 3: feature importance
2.9 Review 1 → Review 2, side by side
2.10 The final phase
2.11 The optional extension (only if asked)

Part 3 — The code, file by file

3.0 The map
3.1 `src/config.py` — the settings file
3.2 `src/preprocessing.py` — loading the data

3.3 `src/base_learners.py` — the four models
3.4 `src/ensemble.py` — the paper's team (the heart of Review 1)
3.5 `src/evaluate.py` — scoring and checking the paper
3.6 `run_all.py` — the one command
3.7 `src/review2.py` — Review 2's experiments A and B
3.8 `src/feature_importance.py` — experiment C
3.9 `src/novelty_cluster_mine_feedback.py` — the optional extension
3.10 `results/` — what each output is
3.11 `app/` — the web application
3.12 `notebooks/`
3.13 `report/` — the report and paper builders
3.14 `presentation/`
3.15 Small files

Part 4 — Questions the panel may ask

Part 5 — The numbers on one page

Part 6 — Tomorrow

The slide script (short version; full notes are inside the PowerPoint)
The demo
Last-minute checklist

How to use this guide tonight

You do not need to know any machine learning before reading this. Read it in this order:

Time	Read	Why
45 min	**Part 1** — machine learning from zero	Every word you will say tomorrow, explained with everyday examples
30 min	**Part 2** — the project story	What you did, in the order you did it, with every number
45 min	**Part 3** — the code, file by file	So you can open any file and say what it does
30 min	**Part 4** — questions the panel may ask	Ready answers, in plain words
10 min	**Part 5** — the numbers on one page	Keep this open during the review
20 min	**Part 6** — tomorrow's plan	The slide script and the demo

If you only have **15 minutes**, read the box below and Part 5.

> **The whole project in five sentences.**
>
> 1. A 2024 research paper says that a team of four machine-learning models, voting with clever weights, can predict a red wine's quality grade from its lab tests with **89.73 %** accuracy.
> 2. I rebuilt that method exactly and tested it fairly (every model tested only on wines it never saw, 100 times over): it reaches **69.01 %**, and a single Random Forest does better at **71.03 %**.
> 3. The reason: the paper's weights come out almost equal (about 0.25 each), so the "team" just averages its members — and the paper's own tables point to the same thing (they imply about **67.5 %**).
> 4. For Review 2, I did the three things my guide asked: **class balancing** (SMOTE finds far more rare wines; macro-F1 **36.92** → **40.92**), **a better result** (learning the weights properly lifts the paper's own team to **70.89 %**), and **feature importance** (**alcohol, sulphates and volatile acidity** drive quality; 4 lab tests give almost all of the accuracy).
> 5. Everything is delivered as a tested **web app**, a **69-page report**, a **10-page conference paper** and **code on GitHub**.

Part 1 — Machine learning from zero

1.1 What is machine learning?

Imagine teaching a child to recognise ripe mangoes. You do not write rules like "if the colour value is above 120 …". You show the child **many mangoes** and say "ripe" or "not ripe" for each. After enough examples, the child can judge a **new** mango alone.

Machine learning is exactly that, for a computer:

- You give it many **examples** (here: wines).
- For each example you give the **answer** (here: the quality grade the tasters gave).
- It finds the pattern that connects the examples to the answers.
- Then it can give an answer for a **new** example it has never seen.

The "pattern" it finds is called a **model**. Finding it is called **training** (or "fitting").

1.2 The words for data

Think of the data as an Excel sheet.

Word	Meaning	In this project
Dataset	The whole sheet	1,599 red wines
Sample / row	One example	One wine
Feature / input / column	Something measured about each example	The 11 lab measurements (alcohol, pH, …)
Label / target / output	The answer we want to predict	The quality grade (3 to 8)
Class	One possible answer, when answers are categories	Grade 3, grade 4, …, grade 8 = six classes

The 11 features are: fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, free sulphur dioxide, total sulphur dioxide, density, pH, sulphates, alcohol.

Some you should be able to explain simply:

- **Alcohol** — % alcohol by volume. Better wines in this data tend to have more.
- **Volatile acidity** — mostly acetic acid; too much tastes like **vinegar**. Worse wines have more.
- **Sulphates** — a preservative additive (it helps protect the wine). Better wines tend to have more.
- **Citric acid** — adds freshness. Slightly more in better wines.
- **Sulphur dioxide (free / total)** — preservative; protects against spoilage.
- **Chlorides** — salt content.
- **Density, pH, residual sugar, fixed acidity** — basic chemistry of the wine.

1.3 Classification or regression?

- **Classification** = the answer is a **category** (“which box does it go in?”). Example: spam or not spam.
- **Regression** = the answer is a **number on a scale** (“how much?”). Example: tomorrow’s temperature.

Wine grade 3–8 could be treated either way. The **base paper treats it as classification** (six classes), so we do too — that is what “reproduce” means. (Treating it as an *ordered* answer is in our future work.)

1.4 Training and testing — the exam rule

If a teacher grades students on the **same questions they practised**, everyone scores 100 % — but that says nothing about whether they learned.

So in machine learning:

- **Training set** = the practice questions. The model learns from these.
- **Test set** = the exam. The model has **never seen** these wines.

We only trust the score on the **test** set.

Overfitting = memorising the practice questions instead of learning the idea. Such a model does great in practice and badly in the exam. Testing on unseen wines is how we catch it.

1.5 Cross-validation — giving every wine a turn as the exam

One split into “practice” and “exam” is a bit lucky or unlucky depending on which wines land where. **Cross-validation** fixes that:

1. Cut the 1,599 wines into **10 equal groups** (“folds”).
2. Round 1: learn from groups 2–10, test on group 1.
3. Round 2: learn from groups 1 and 3–10, test on group 2.
4. … until each group has been the test group once.

Now **every wine has been tested once**, by a model that never saw it. That is **10-fold cross-validation**.

Stratified means each group has the **same mix of grades** as the whole data. This matters a lot here, because grade 3 has only 10 wines — without stratifying, some groups might get none.

Repeated 10 times: we reshuffle the wines and do the whole 10-fold thing again, ten times (with shuffle seeds 0 to 9). So each wine is predicted 10 times → **10 × 1,599 = 15,990 predictions per model**. The paper says it used “10 runs of 10-fold cross-validation” (100 experiments), so this matches it.

We then compute the scores over all 15,990 predictions together (“**pooled**”).

> **Say it like this:** “Every model was tested only on wines it had never seen. I split the data into ten groups with the same grade mix, gave each group a turn as the test set, and repeated that ten times with different shuffles — 100 fold-runs, as in the paper.”

1.6 Data leakage — peeking at the exam answers

Leakage = information from the test wines sneaking into training. It makes scores look better than they really are.

A subtle example that **this project fixed**: scaling. If you compute "the smallest and largest alcohol value" using **all** 1,599 wines *before* splitting, then the test wines have already influenced how the training data was scaled. That is a small peek at the exam.

Fix: put the scaler **inside a pipeline** together with the model. A pipeline is a chain: "first scale, then model". When cross-validation trains the pipeline on the training groups, the scaler learns its min and max **only from those groups**, and just applies them to the test group.

The same rule is used everywhere in the project: - **SMOTE** (Part 1.10) only sees the training wines. - **Feature selection** (choosing the top-k measurements) is done inside each training fold. - The **temperature** in the improved ensemble is chosen inside each training fold.

1.7 Scaling (min–max normalisation)

The features have very different units: density is about 0.99, total sulphur dioxide can be 289. Some models (like SVM and neural networks) measure distances, so a big-number feature would drown out a small-number feature.

Min–max scaling squeezes every feature into the range 0 to 1:

> new value = (value − smallest) ÷ (largest − smallest)

Tiny example: alcohol ranges 8.4 to 14.9. A wine with 10.4 % alcohol becomes (10.4 − 8.4) ÷ (14.9 − 8.4) = 2.0 ÷ 6.5 ≈ **0.31**.

The paper does this step too (its Fig. 2), so we keep it — but inside the pipeline.

1.8 How we score a model

The confusion matrix

A table: rows = the **true** grade, columns = the **predicted** grade. The diagonal = correct answers. Everything off the diagonal = mistakes. In this project most mistakes land in the **neighbouring** grade (a 6 called a 5), which makes sense.

The four counts, for one class (say grade 4)

- **TP (true positive)** — a real grade-4 wine, called grade 4. ✔
- **FN (false negative)** — a real grade-4 wine, called something else. We **missed** it.
- **FP (false positive)** — some other wine, wrongly called grade 4. A **false alarm**.
- **TN (true negative)** — not grade 4, and not called grade 4.

The measures

Measure	Plain meaning	Formula
Accuracy	Of all wines, what share got the right grade?	correct ÷ all
Recall (= sensitivity)	Of the real grade-4 wines, how many did we **find**?	TP ÷ (TP + FN)
Precision	Of the wines we **called** grade 4, how many really are?	TP ÷ (TP + FP)
F1	One number balancing precision and recall for a class	2·P·R ÷ (P + R)
Macro-F1	Average of the F1 of all **six** grades, each counted **equally**	mean of six F1s
Balanced accuracy	Average of the recall of all six grades, each counted equally	mean of six recalls
Specificity	Of the wines that are NOT grade 4, how many did we correctly not call grade 4?	TN ÷ (TN + FP)

Why accuracy alone is misleading here

82.5 % of the wines are grade 5 or 6. A lazy model that **always says "5"** is right 681 ÷ 1,599 = **42.6 %** of the time — without ever finding a single grade-3, 4, 7 or 8 wine. Accuracy mostly measures how well you do on grades 5 and 6.

Macro-F1 counts each grade equally, so a model only scores well if it also handles the rare grades. That is why Review 2 uses macro-F1 as the **main ("primary") measure** — and this choice was **written into the code before** any Review-2 experiment was run (so nobody can say we picked the measure that made us look good).

The paper's "precision" mistake (worth knowing)

The paper calls its per-class measure "precision", but its own worked example divides by the number of wines that **truly** belong to the grade — that is **recall**, not precision. We checked: 4 of the paper's 6 per-class numbers reproduce when computed as recall, only 1 as precision. So when we compare with the paper class by class, we use recall (their actual definition), and we report real precision separately. Say it politely: *"the paper's per-class metric is labelled precision but computed as recall; we compare like with like."*

1.9 Class imbalance — why the rare grades get ignored

Grade	3	4	5	6	7	8
Wines	10	53	681	638	199	18

A model learns by reducing its mistakes. Getting all 10 grade-3 wines wrong costs it almost nothing compared with getting grade 5 right. So, left alone, models **ignore the rare grades**. In our Review-1 ensemble, **0 %** of grade-3 wines and **0.6 %** of grade-4 wines were found.

Two fixes:

1. **Class weights** — make a mistake on a rare grade **cost more** during training. "Balanced" weights: each wine's weight = N ÷ (number of classes × wines in its class). So a grade-3 wine gets weight 1,599 ÷ (6 × 10) ≈ 26.7, and a grade-5 wine 1,599 ÷ (6 × 681) ≈ 0.39 — the grade-3 wine counts about 68 times more. (SVM and Random Forest have a `class_weight` setting; Gradient Boosting and MLP do not, so we give them the same weights wine by wine through a small wrapper called `BalancedSampleWeight`.)
2. **SMOTE** (Synthetic Minority Over-sampling TEchnique) — **create new examples** of the rare grades. Take a rare wine, pick one of its 5 most similar rare wines, and make a new "in-between" wine somewhere on the line joining them: new = wine + λ × (neighbour − wine), with λ a random number between 0 and 1. Repeat until every grade has as many wines as the biggest one. Like mixing two shades of red paint to get a new red in between.

Important: SMOTE runs **inside each training fold, after scaling**. The test wines are never copied or blended into training. (If SMOTE ran on all data first, a blend of a test wine could appear in training — leakage.)

The honest trade-off: balancing does not create new information. It moves the model's attention from the crowded grades to the rare ones. Rare grades get found more often, but some ordinary grade-5/6 wines now get called rare by mistake. Result: **macro-F1 up, accuracy a little down.**

1.10 The models, one by one (with everyday pictures)

Decision tree (the building block)

The game "20 questions": *Is alcohol above 11.5? → yes → Are sulphates above 0.65? → yes → grade 7.* A tree is a list of yes/no questions learned from data. Easy to understand, but a single tree is unstable and overfits easily.

Random Forest (RF) — a crowd of trees

Train **300 trees**, each on a **random sample** of the wines (a "bootstrap sample") and looking at a **random subset of features** at each question. Each tree votes; the forest averages their votes (their probabilities). A crowd of slightly different trees is much more reliable than one tree — like asking 300 people instead of one. In our project RF is the **strongest single model (71.03 %)**. Setting used: `class_weight="balanced_subsample"` (balanced weights recomputed for each tree's sample).

Gradient Boosting (GB) — each new member fixes the last one's mistakes

Train small trees **one after another**. Each new tree focuses on the mistakes the previous trees still make, and is added with a small step size (the "learning rate", 0.1). 200 rounds. Like a student who, after each test, studies only the questions they got wrong. **Used in place of the paper's RNN** (both improve step by step; RNN is outside our syllabus).

SVM (Support Vector Machine) — the widest street

Imagine wines as dots on a map, coloured by grade. An SVM draws the boundary between groups so that the **gap (the "street") on both sides is as wide as possible** — that generalises best. The **RBF kernel** lets the boundary bend (curves instead of straight lines). Settings: C = 10 (how strictly to avoid

mistakes), gamma = “scale”. An SVM gives only a decision, not probabilities, so we wrap it in **calibration** (Platt scaling) to turn its scores into probabilities, learned inside the training fold.

MLP (Multilayer Perceptron) — a small neural network

Layers of tiny calculators (“neurons”). Each neuron multiplies its inputs by weights, adds them, and passes the result through a simple function (ReLU: “if negative, make it 0”). Our MLP: 11 inputs → 64 neurons → 32 neurons → 6 outputs (one probability per grade, via “softmax”). It learns its weights by **back-propagation**: make a prediction, measure the error, and nudge every weight a little in the direction that reduces the error; repeat thousands of times. **Used in place of the paper’s ELM.**

What the paper used and why we replaced two

- **RNN (Recurrent Neural Network)** — a network made for **sequences** (sentences, time series), with memory of previous steps. Deep learning → outside the course syllabus.
- **ELM (Extreme Learning Machine)** — a network whose hidden weights are **random and never trained**; only the last layer is solved directly with a formula. Not trained by back-propagation → outside the syllabus.
- So: RNN → Gradient Boosting, ELM → MLP. SVM and RF are kept exactly.

Extra models tried in Review 2

- **ExtraTrees (Extremely randomised trees)** — like Random Forest, but the split points are also chosen randomly. 500 trees.
- **HistGB (Histogram Gradient Boosting)** — a fast version of gradient boosting that first groups values into bins (like LightGBM).
- **Logistic Regression** — a simple model that turns a weighted sum into probabilities; used as the “combiner” in stacking.

1.11 Ensembles — the wisdom of the crowd

An **ensemble** is a team of models that vote. It helps **only if** the members make **different** mistakes (that is “diversity”): then one member’s mistake is out-voted by the others being right.

Two ways to vote: - **Hard vote** — each model says one grade; count the (weighted) votes. - **Soft vote** — each model gives a **probability for every grade**; add the (weighted) probabilities; the grade with the biggest total wins. A confident strong model can then outweigh several unsure weak ones.

The paper’s formula (its Eq. 1) is a weighted **sum** of the models’ outputs, which most naturally reads as the **soft** vote, so the soft vote is our headline. We also report the hard vote (69.93 %) to be transparent.

The paper’s weights — in plain words

Each model gets a weight made of two halves:

1. **Accuracy weight** (paper’s Eqs. 5–6): compute the model’s error rate ε (share of wrong answers). Take 1 ÷ ε, so better models get bigger numbers. Divide by the total so the four weights add up to 1.
2. **Diversity weight** (Eqs. 7–9): write each model’s mistakes as a long list of 0s and 1s (1 = wrong on that wine) — the “error vector”. Measure the distance between one model’s list and each of the other

models' lists, add those distances up (D), and divide by the total. A model whose mistakes are **different from the others'** gets a bigger weight.

3. **Final weight** (Eq. 4): the average of the two halves.

The catch (your key finding): good models have similar error rates (say 29 % and 38 %), so 1 ÷ ε gives similar numbers (3.4 and 2.6). The distances are also similar. So **all four weights end up close to 0.25** — ours: 0.237 / 0.270 / 0.257 / 0.237; the paper's own: 0.252 / 0.229 / 0.217 / 0.303. With equal weights, a vote is just an **average**. And an average of one strong model (RF) and three weaker ones **lands between them** — above the weak ones, below the strong one. That is exactly what we got.

> **Analogy:** a cricket team where the best batter and the weakest batter get the same say in every decision. The team's average is lower than its best player.

The temperature fix

Raise each half of the weight to a power **T** before dividing by the total. T = 1 is exactly the paper. T = 10 or 25 makes the differences much bigger, so the best model (RF) gets more of the vote. Think of turning up the contrast on a photo.

1.12 Stacking — let a model learn how to combine

Instead of hand-made weights, train a **second-level model** (logistic regression) that looks at the four models' probabilities and learns the best way to combine them. It is trained on predictions the members made for wines they did not see (an inner 5-fold split), so it does not cheat. In our data plain stacking **learned the majority bias back** (high accuracy, low macro-F1); a balanced combiner went the other way.

1.13 Feature importance — which measurements matter?

1. **Permutation importance** (our main method): take a trained model, **scramble one measurement** in the test wines (shuffle that column so it no longer matches the wines), and see **how much the score drops**. Big drop = the model relied on it. Like testing how much a recipe needs salt by leaving the salt out. Done on unseen test folds, 20 shuffles per feature per fold, measured in macro-F1 points.
2. **MDI (Mean Decrease in Impurity)**: Random Forest keeps a score of how much each feature's questions helped it split the wines cleanly. Free to compute, but known to **favour features with many different values**, so we only use it as a cross-check.
3. **Spearman correlation (ρ)**: does more of a feature go with better (+) or worse (–) wine? Gives the **direction**, which the other two do not. It ranks values instead of using them raw, so it is not thrown off by extreme values.

Then a practical question: **how many lab tests are needed?** Retrain RF using only the top 1, 2, 3, 4, 6, 8 or 11 measurements, chosen **inside each training fold**.

1.14 A little statistics

- **Paired comparison**: every model was tested on the same splits in each of the 10 repeats, so for two models we can compare repeat by repeat and count **how many of the 10 repeats each one wins**.

- **Wilcoxon signed-rank test**: a standard test for “is this paired difference consistently positive or just noise?”. It gives a **p-value**: small (below 0.05) means “unlikely to be just luck”.
- **Honest caveat**: the 10 repeats reuse the **same 1,599 wines** with different shuffles, so they are not fully independent; the p-values are therefore a little optimistic. The **win count** (e.g. 10 of 10) is the plainer evidence. A “corrected resampled t-test” (Nadeau & Bengio, 2003) would be more exact — that is in future work.

Part 2 — The project story

2.1 The problem

Wine retailers decide prices, stock and promotions by quality. Quality comes from a tasting panel: slow, costly, partly subjective. Lab tests are cheap, objective and available early. **Goal: predict the tasters' grade from the 11 lab tests.**

2.2 The dataset

- UCI Wine Quality, **red** part: **1,599** Portuguese "Vinho Verde" red wines, collected by **Cortez et al. (2009)**.
- 11 lab measurements; quality = median grade of at least three tasters on a 0–10 scale. Only grades **3–8** occur.
- **Very imbalanced**: grades 5 and 6 = 82.5 %; grades 3 and 8 together = 1.75 %.
- No missing values. 240 rows are exact duplicates of another row; we keep all rows, as the paper does.
- Strongest single links with quality (Pearson r): alcohol +0.48, volatile acidity −0.39, sulphates +0.25, citric acid +0.23.

2.3 The base paper

Ma, R., Mao, D., Cao, D., Luo, S., Gupta, S., & Wang, Y. (2024). From vineyard to table: Uncovering wine quality for sales management through machine learning. *Journal of Business Research*, 176, 114576. Elsevier; indexed in Scopus and SSCI. Authors from the University of Greenwich, Nottingham Trent, Newcastle, Sheffield and a Chinese power-company research institute.

What it does: four models (SVM, RNN, ELM, RF) → each gets an accuracy weight and a diversity weight → weighted vote. Claims:

	SVM	RNN	ELM	RF	Ensemble
Accuracy (paper)	67.30	61.43	55.77	80.71	**89.73**
Final weight (paper's Table 8)	0.2516	0.2286	0.2168	0.3030	—

It also says alcohol, sulphates, volatile acidity and citric acid matter most. Typical published accuracies for this six-grade task are in the low 60s to low 70s, so 89.73 % is unusually high — which is why checking it is interesting.

2.4 Review 1 — the reproduction

What I built: the four models (SVM and RF as in the paper; Gradient Boosting for the RNN and MLP for the ELM), the paper's accuracy + diversity weights exactly as its equations say, soft and hard voting, min–max scaling inside a pipeline, stratified 10-fold × 10 repeats.

Results (15,990 pooled predictions each):

Rank	Model	Accuracy
1	Random Forest	**71.03 %**
2	Ensemble, hard vote	69.93 %
3	**Ensemble, soft vote (headline)**	**69.01 %**
4	Gradient Boosting (for RNN)	67.67 %
5	SVM	61.95 %
6	MLP (for ELM)	61.35 %

- The paper's central claim **does not reproduce**: our team is **2.02 points below** Random Forest, not 9 above.
- Per grade (recall, the paper's "precision"): grade 3 **0.0 %**, 4 **0.6 %**, 5 **81.7 %**, 6 **69.9 %**, 7 **49.9 %**, 8 **11.1 %**. Grades 5 and 6 are predicted far better than in Cortez et al. (67.5 / 57.7), so the *shape* reproduces; the rare grades are almost never found.

Why — three checks from the paper's own tables (in `src/evaluate.py`):

1. **Different test set.** The paper's confusion matrix (Table 9) adds up to only **491 wines** = 30.7 % of the data. That looks like one hold-out split, not 100 cross-validation runs over all 1,599 wines.
2. **Its own arithmetic.** The correct predictions in that table are 403 of 491 = **82.08 %**, which is **7.65 points below** the 89.73 % stated in the text.
3. **Flat weights.** Applying the paper's own weights to its own model accuracies gives **67.52 %**. Our 69.01 % is 1.49 points from that; the paper's claim is 22.21 points from it. (A good team *can* beat this weighted average when its members err differently — so this is a strong hint, not a proof.)

Caveat you must always say: the Table 9 numbers were read by eye from the PDF, so we say "**we could not reconcile these figures**", never "the paper is wrong".

Our explanation: the weights are almost equal → the team averages → it lands between its members. Our weights differ from 0.25 by at most 0.02; the paper's by at most 0.053.

2.5 What the guide asked after Review 1

> "Class balancing: yet to be done; improve result; work on feature importance."

2.6 Review 2 — change 1: class balancing

Four schemes, **same four models, same weighted vote, same folds** — only the balancing changes:

Scheme	Accuracy	Bal. acc.	Macro-F1	Grade 3	Grade 4	Grade 7	Grade 8
No balancing	68.82	35.28	36.64	0.0	0.2	49.0	11.1
Review 1 (SVM + RF weighted)	69.01	35.52	36.92	0.0	0.6	49.9	11.1
Class weights, all four	68.43	37.94	38.77	0.0	7.7	61.9	12.2
SMOTE	63.86	41.44	**39.34**	10.0	26.4	64.5	15.6

(The grade columns are recall: % of that grade's wines found.)

- SMOTE gives the best macro-F1 for the ensemble; grade 4: **0.6 %** → **26.4 %**, grade 7: **49.9 %** → **64.5 %**.
- Price: accuracy **–5.15 points** (some ordinary wines now called rare).
- Macro-F1 per model (none / Review-1 / class weights / SMOTE): SVM 29.30 / 28.01 / 28.01 / 34.24; RF 38.18 / 38.09 / 38.09 / **40.92**; GB 36.72 / 36.72 / 37.78 / 38.08; MLP 30.07 / 30.07 / 29.00 / 34.21. SMOTE helps every model; class weights hurt the MLP (its training gets unstable when 10 wines carry huge weights).
- **Regression check:** inside the new code, the "Review 1" scheme reproduces 69.01 % to every digit — proof that Review 2 is comparable with Review 1.

2.7 Review 2 — change 2: improving the result

Step 1 — test the explanation (diagnostic only). Sharpen the weights with a temperature T (T = 1 = paper):

T	1	2	5	10	25	100
Accuracy, sharpen the accuracy half	69.01	69.22	69.90	70.78	**71.22**	71.23
Biggest weight	0.270	0.293	0.364	0.466	0.590	0.623
Accuracy, sharpen both halves	69.01	69.20	69.82	70.59	71.08	70.41

From T = 25 the team **overtakes Random Forest (71.03 %)** → the flat-weights explanation **holds**. Sharpening *both* halves eventually backfires: the diversity half gives more weight to the model whose mistakes are most different — usually the **weakest** one. This sweep is **only a test**, because here the weights are computed from the same predictions they are scored on — so it is never reported as "the improved result".

Step 2 — the proper fix: learned temperature. Inside **each outer training fold**, run a small 5-fold cross-validation to choose T (from 1, 2, 5, 10, 25, 100) and whether to sharpen one or both halves,

scoring by macro-F1. Then retrain on the whole training fold and predict the test fold. The test fold never influences the choice → a legitimate result.

- Same four models: accuracy **69.01 % → 70.89 %**, macro-F1 **36.92 → 38.08**. Better than the paper's weights in **10 of 10 repeats** on both measures.
- Now level with Random Forest (70.89 % vs 71.03 %): the learned-temperature team wins 4 of 10 repeats on accuracy and 6 of 10 on macro-F1 — a statistical tie (p = 0.44 and 0.92).
- Left to choose, the inner test picks a **sharp T (≥ 10) in 99 % of folds** (T = 100 in 45 folds, 25 in 36, 10 in 18, 5 in 1) → the data itself agrees the paper's T = 1 is too flat.
- With SMOTE members: macro-F1 **40.87**, accuracy 66.63 %.

Step 3 — other routes tried:

- **Stacking** (logistic-regression combiner on the 4 models): 70.92 % accuracy but macro-F1 only **34.96** — it learned the majority bias back. **Balanced combiner**: 59.20 % accuracy, the best balanced accuracy of all (50.03 %), finds 48 % of grade-3 and 53 % of grade-8 wines.
- **Stronger models**: ExtraTrees alone 70.97 %; HistGB 68.89 %; the paper's weights on SVM + RF + ExtraTrees + HistGB 70.79 %; stacking those four 70.93 %.

All 28 candidates (scored on the same 100 fold-runs) — no model wins both measures:

- **Best macro-F1: Random Forest + SMOTE — 40.92** (accuracy 67.94 %, balanced accuracy 42.07 %). It beats the Review-1 team on macro-F1 in **10 of 10** repeats (mean +3.97, Wilcoxon p = 0.002) and loses on accuracy in 9 of 10 (mean −1.08).
- Runner-up: learned-temperature ensemble with SMOTE members, 40.87 — a tie with the winner (p = 0.85).
- **Best accuracy: Random Forest (Review-1 setup) — 71.03 %.**

Honest disclosures (say them before you are asked): - The winner is picked on the same cross-validation that scores it → a small optimistic bias. - Two candidates (learned temperature, balanced stacking) were added after a 1-repeat trial run. Their settings are still chosen inside each training fold, and the reported numbers come from the full 10-repeat run.

2.8 Review 2 — change 3: feature importance

Rank	Measurement	Permutation (best model)	Permutation (RF)	MDI (RF)	Spearman ρ	Paper's key?
1	alcohol	7.45	8.48	0.135	+0.48	yes
2	sulphates	7.28	6.49	0.119	+0.38	yes
3	volatile acidity	4.44	4.86	0.129	–0.38	yes
4	chlorides	2.33	1.05	0.093	–0.19	
5	total sulphur dioxide	1.66	2.27	0.089	–0.20	
6	citric acid	0.56	1.39	0.079	+0.21	yes
7–11	fixed acidity, residual sugar, density, pH, free SO_2	≈ 0 or below				

(Permutation numbers = macro-F1 points lost when the measurement is scrambled.)

- **Top three: alcohol, sulphates, volatile acidity.** Three methods agree (rank agreement 0.74–0.89).
- Direction: more **alcohol, sulphates, citric acid** → **better**; more **volatile acidity** → **worse** (vinegar). Matches wine chemistry — a good sanity check.
- **3 of the paper's 4 key attributes confirmed**; citric acid ranks 6th.
- Negative values mean "no usable signal for this model": often because another feature carries the same information (e.g. density overlaps with alcohol).

How many lab tests are needed? (Random Forest, features chosen inside each training fold)

Tests	1	2	3	4	6	8	11 (all)	Paper's 4
Accuracy %	28.98	58.93	66.25	**69.27**	70.21	70.79	71.03	67.55
Macro-F1	18.51	32.65	36.77	**38.46**	38.35	38.23	38.09	37.68

Four tests (alcohol, volatile acidity, sulphates, chlorides) give **97 %** of the accuracy of all eleven — and the best macro-F1. Business reading: a cheaper test panel for routine screening.

2.9 Review 1 → Review 2, side by side

Measure (%)	Review-1 ensemble	Review-1 Random Forest	Review-2 best (RF + SMOTE)	Change vs ensemble
Accuracy	69.01	71.03	67.94	–1.08
Balanced accuracy	35.52	36.40	42.07	+6.55
Macro-F1	36.92	38.09	**40.92**	**+4.00**
Grade 3 found	0.00	0.00	4.00	+4.00
Grade 4 found	0.57	0.00	21.89	+21.32
Grade 5 found	81.66	81.97	78.00	–3.66
Grade 6 found	69.87	74.29	64.18	–5.69
Grade 7 found	49.90	51.06	65.43	+15.53
Grade 8 found	11.11	11.11	18.89	+7.78

2.10 The final phase

- **Web application** (Streamlit): 7 pages — Overview, Dataset, Review 1, Class balancing, Improving the result, Feature importance, Predict a wine. The predict page has 11 sliders (or "start from a typical grade-X wine"), shows the grade and the probability of each grade from **two models** (Random Forest and Random Forest + SMOTE), and has a **batch CSV** upload/download. It uses the project's own code, and **15 automated checks** pass.
- **Project report**: 69 pages in the SASTRA format, 9 chapters, 33 references (every DOI checked).
- **Conference paper**: 10 pages.
- **GitHub**: github.com/Surianandhan/wine-quality-ml-project (code, data, results; not the paper PDF — it is copyrighted).
- **Colab notebook**: runs the whole project in a browser.

2.11 The optional extension (only if asked)

`src/novelty_cluster_mine_feedback.py`, run with `python run_all.py --with-novelty`. Coursework beyond the reproduction: - **Clustering** (K-means, Ward) on the 11 measurements only: best K-means with k = 3, silhouette 0.211; clusters follow quality only loosely. - **Outlier detection** (Isolation Forest, LOF): grade-3 wines are flagged as unusual about twice as often as grades 5/6 (20 % vs ~7–10 %) — they are genuinely atypical, not just few. - **Cluster ID as a 12th feature**: did **not** help the rare grades (a negative result, reported as such). - **Association rules** (FP-growth): rules for "high quality" recover all four of the paper's key attributes plus others.

Part 3 — The code, file by file

3.0 The map

```
wine-quality-ml-project/
├── run_all.py                  ← the ONE command that runs every experiment
├── src/                        ← the machine-learning code
│   ├── config.py               settings + the paper's published numbers
│   ├── preprocessing.py        load the wine data
│   ├── base_learners.py        the four models (each = scaler + model)
│   ├── ensemble.py             the paper's weights + votes + the 10×10 CV loop
│   ├── evaluate.py             scores, confusion matrices, checks on the paper
│   ├── review2.py              class balancing, temperature, stacking, comparisons
│   ├── feature_importance.py   permutation importance, top-k tests
│   └── novelty_cluster_mine_feedback.py   optional extension
├── data/raw/                   winequality-red.csv (+ white, unused)
├── results/                    every table (CSV) and figure (PNG) the experiments write
│   ├── tables/, figures/       Review 1
│   └── review2/tables, figures Review 2
├── app/                        the web application
│   ├── streamlit_app.py        the 7 pages
│   ├── test_app.py             the 15 automated checks
│   └── requirements.txt        exact library versions for the cloud
├── notebooks/                  Colab notebook (+ the script that builds it)
├── report/                     scripts that build the report and the paper
├── presentation/               the three review decks and these notes
└── README.md, HOW_TO_RUN.md, DEPLOY.md, requirements.txt, .gitignore
```

How data flows:

```
winequality-red.csv
   → preprocessing.load_red_wine_xy()        (X = 11 measurements, y = grade)
   → ensemble.run_repeated_cv()              (10 folds × 10 repeats, 4 models, weights,
votes)
        uses base_learners.get_base_learners()
   → evaluate.evaluate_all()                 (scores, confusion matrices, paper checks)
   → results/tables/*.csv, results/figures/*.png         ← Review 1
run_all.py --review2
   → review2.py experiments A and B, feature_importance.py experiment C
   → results/review2/tables/*.csv, figures/*.png         ← Review 2
Everything else (app, report, paper, slides) READS those CSVs and PNGs.
```

That last line is the key design idea: **one source of truth**. No number is typed by hand anywhere, so the slides, report, paper and app can never disagree.

3.1 `src/config.py` — the settings file

One place for every setting, so nothing is repeated across files.

- **Paths**: where the data is, where results go (`results/tables`, `results/review2/...`), and a separate scratch folder for quick runs so they never overwrite the real results.

- `FEATURE_COLUMNS` (the 11 names) and `TARGET_COLUMN = "quality"`.
- **Protocol**: `RANDOM_STATE = 42` (fixed randomness → same results every run), `N_SPLITS = 10`, `N_REPEATS = 10`, `QUICK_N_REPEATS = 2`.
- **The paper's published numbers**, used only for comparison: per-class values of Cortez et al. and Ma et al., the 89.73 %, the Table 9 confusion matrix (with a comment saying it was typed by eye from the PDF), the base-model accuracies (67.30, 61.43, 55.77, 80.71) and the Table 8 weights.
- **Review 2 settings**: `REVIEW2_PRIMARY_METRIC = "macro_f1"` (fixed before any experiment), `SMOTE_K_NEIGHBORS = 5`, `WEIGHT_TEMPERATURES = [1, 2, 5, 10, 25, 100]`, `FEATURE_SUBSET_SIZES = [1, 2, 3, 4, 6, 8, 11]`, `PERMUTATION_REPEATS = 20`, and `PAPER_KEY_ATTRIBUTES`.

3.2 `src/preprocessing.py` — loading the data

- `load_red_wine()` reads the CSV (it uses `;` as separator, as UCI ships it).
- `get_features_target()` splits it into **X** (the 11 numbers per wine) and **y** (the grade).
- `load_red_wine_xy()` — **the loader everything uses**. It returns **raw, unscaled** X, on purpose: scaling happens later, inside each model's pipeline, fitted on the training fold only (no leakage).
- `minmax_normalize()` / `load_normalized_red_wine()` — scale everything at once; used **only** by the optional clustering extension, which never splits into train/test, so leakage is not an issue there.
- `train_test_split_data()` — a single 80/20 split kept for illustration; the real results never use it.

3.3 `src/base_learners.py` — the four models

- The long comment at the top explains the substitutions (RNN → Gradient Boosting, ELM → MLP) and the leakage fix.
- `_pipe(model)` builds `Pipeline([MinMaxScaler, model])` — "scale, then model" as one unit.
- `get_base_learners()` returns the four pipelines:
  - **SVM**: `SVC(kernel="rbf", C=10, gamma="scale", class_weight="balanced")` inside `CalibratedClassifierCV` (turns its scores into probabilities with Platt scaling, inside the training fold).
  - **RF**: `RandomForestClassifier(n_estimators=300, class_weight="balanced_subsample")`.
  - **MLP**: `MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=2000)`. No early stopping, because it would hold back 10 % of a training fold that has only 9 grade-3 wines.
  - **GradBoost**: `GradientBoostingClassifier(n_estimators=200)`.
- `cross_val_predictions()` / `cross_val_accuracy_scores()` — small helpers to cross-validate one model at a time.

3.4 `src/ensemble.py` — the paper's team (the heart of Review 1)

- The top comment writes out the paper's Eqs. (1)–(9) and explains **soft vs hard** reading of Eq. (1).
- `_error_vector(y_true, y_pred)` → the 0/1 list of mistakes (paper's g and e). 1 = wrong.
- `compute_weights(error_vectors)` → the paper's weights exactly:
  - error rate = mean of the 0/1 list (Eq. 6);

    - accuracy weight = (1 ÷ error) ÷ sum (Eq. 5);
    - diversity D = sum of distances to the other lists (Eq. 8), normalised (Eq. 9);
    - final = ½ × (accuracy + diversity) (Eq. 4).
- `weight_concentration(weights)` → how far the weights are from 0.25 (the "flat weights" diagnostic).
- `weighted_vote()` (hard) and `weighted_soft_vote()` (soft: add weight × probabilities, take the biggest).
- `run_repeated_cv(X, y)` — **the main loop**:
    1. for each repeat r = 0…9: make a stratified 10-fold split with shuffle seed r;
    2. for each model: `cross_val_predict(..., method="predict_proba")` → probabilities for every wine from a model that never saw it;
    3. turn probabilities into predicted grades; build each model's error list;
    4. compute the paper's weights from this repeat's mistakes;
    5. soft vote and hard vote;
    6. store everything; at the end, join ("pool") all 10 repeats → 15,990 predictions per model. All models use the same folds in a repeat (needed for the diversity weight, which compares mistakes wine by wine).

3.5 `src/evaluate.py` — scoring and checking the paper

- `per_class_metrics()` → precision, recall, specificity per grade, plus the paper's "precision" (which is really recall — explained in the file's top comment).
- `specificity_per_class()`, `overall_accuracy()`, `confusion_matrix_df()`, `save_model_evaluation()` → write `confusion_matrix_*.csv` and `metrics_*.csv`.
- `build_benchmark_comparison()` + `plot_benchmark_comparison()` → our per-grade results next to Cortez (2009) and Ma (2024) → `benchmark_comparison.csv`, `accuracy_comparison.png`.
- `paper_table9_reconciliation()` → **check 1 and 2**: adds up the paper's Table 9 (491 wines), counts the diagonal (403 → 82.08 %), and tests whether the paper's per-class numbers match recall (column-wise) or precision (row-wise).
- `paper_internal_consistency_check()` → **check 3**: paper's weights × paper's accuracies = 67.52 %; compares with 89.73 % and with our 69.01 %.
- `build_accuracy_summary()` → the model ranking table `model_accuracy_summary.csv`.
- `evaluate_all()` → runs all of the above for every model.

3.6 `run_all.py` — the one command

```
python run_all.py                 Review 1 (10 × 10)          ~2 minutes
python run_all.py --quick         same code, 2 repeats         ~30 seconds, separate
folder
python run_all.py --review2       + Review 2 experiments       ~45–60 minutes
python run_all.py --with-novelty  + the optional extension
```

- `step3_reproduction()` → runs `run_repeated_cv` + `evaluate_all`, prints the ranking, the weight diagnostic and the paper checks.
- `step4_novelty()` → the optional extension.

- `step5_review2()` → experiments A (balancing), B1 (temperature sweep), B2/B3/B4 (stacking, stronger models, learned temperature), the 28-candidate comparison, the headline table, confusion matrices, the paired comparison, then C (feature importance). It also runs a **regression check**: the "Review 1" setting inside the new code must give exactly the Review-1 accuracy, or it prints "MISMATCH".
- `main()` reads the flags. `--quick` writes to `results/_quick_run/` so it can never overwrite the real numbers.

3.7 `src/review2.py` — Review 2's experiments A and B

The long comment at the top is a summary of Review 2 — worth reading once.

Building blocks: - `BalancedSampleWeight` — a small wrapper that gives Gradient Boosting and MLP "balanced" weights wine by wine (they have no `class_weight` option). - `make_paper_learners(balancing)` — the same four Review-1 models under one scheme: `"none"`, `"partial"` (= Review 1), `"class_weight"`, or `"smote"` (an imbalanced-learn pipeline: scaler → SMOTE → model, so SMOTE only touches training wines). - `make_strong_learners()` — ExtraTrees (500 trees) and HistGB, both class-weighted. - `make_stack(members, balanced_combiner)` — `StackingClassifier` with a logistic-regression combiner and its own inner 5-fold split. - `tempered_weights(errors, T, sharpen)` — the paper's weights with the temperature T (T = 1 gives exactly the paper). - `PaperWeightedEnsemble` — the paper's team as a normal scikit-learn model. When given several temperatures, its `fit()` runs an inner 5-fold cross-validation on the training data, tries every T and both sharpen modes, keeps the best by macro-F1 (ties → smaller T, i.e. closer to the paper), then trains the four models on the full training fold. This is the **learned-temperature ensemble**. - `make_learned_temperature(balancing)` — builds that ensemble.

Experiments: - `collect_oof_probas()` — out-of-fold probabilities on **exactly the Review-1 folds** (same seeds), so results line up wine for wine. - `score()` — accuracy, balanced accuracy, macro-F1, weighted F1 and recall per grade. - `experiment_a_class_balancing()` → `A_class_balancing.csv` + figure. - `experiment_b1_temperature()` → the diagnostic sweep → `B1_weight_temperature.csv` + figure. - `experiment_b2_b3_improvements()` → stronger models, mixed ensemble, three stacks, learned temperature (with `_oof_learned_temperature()`, which also records which T each fold chose → `B4_chosen_temperatures.csv`). - `build_model_comparison()` → all 28 candidates ranked by macro-F1 → `model_comparison.csv`. - `headline_table()` → Review-1 vs Review-2 table + `headline_best_model.txt`. - `per_repeat_scores()` + `paired_comparison()` → wins out of 10 and Wilcoxon p → `paired_comparison.csv`. - `save_confusion()` → confusion matrices of the Review-1 team and the Review-2 best.

3.8 `src/feature_importance.py` — experiment C

- `_one_fold()` + `permutation_importance_cv()` → for each of the 10 folds: train on 9 folds, then use scikit-learn's `permutation_importance` on the test fold (20 shuffles per feature), measured in macro-F1 and accuracy; also collects RF's MDI.
- `direction_of_effect()` → Spearman ρ of each feature with quality.
- `feature_subset_curve()` → for k = 1, 2, 3, 4, 6, 8, 11: a pipeline of scaler → `SelectFromModel` (a 200-tree forest picks the top k, **inside the training fold**) → 300-tree forest; plus the paper's four attributes. Records which features were picked most often.

- `run_feature_importance()` → runs everything, writes `C_feature_importance.csv`, `C_ranking_agreement.csv`, `C_paper_crosscheck.csv`, `C_feature_subset.csv`, `C_selection_frequency.csv` and the three figures.

3.9 `src/novelty_cluster_mine_feedback.py` — the optional extension

`run_clustering()` (K-means + Ward, k = 3–8, best by silhouette), `run_outlier_detection()` (Isolation Forest + LOF, 10 % contamination), `augment_with_cluster_feature()` + `compare_cluster_feedback()` (cluster ID as a 12th feature), and the FP-growth functions (`discretize_features`, `mine_rules`, `filter_high_quality_rules`, `crosscheck_with_paper_figures`). Runs only with `--with-novelty`.

3.10 `results/` — what each output is

Review 1 (`results/tables/`) - `model_accuracy_summary.csv` — the ranking (71.03 / 69.93 / 69.01 / …). - `benchmark_comparison.csv` — per-grade, ours vs Cortez vs Ma. - `metrics_<model>.csv`, `confusion_matrix_<model>.csv` — per model. - `paper_table9_reconciliation.csv`, `paper_table9_per_class_check.csv`, `paper_internal_consistency_check.csv` — the three checks. - (Extension: cluster, outlier and rule tables.)

Review 2 (`results/review2/tables/`) - `A_class_balancing.csv`, `B1_weight_temperature.csv`, `B2_B3_improvements.csv`, `B4_chosen_temperatures.csv` - `model_comparison.csv` (all 28), `headline_review1_vs_review2.csv`, `headline_best_model.txt` - `paired_comparison.csv`, `per_repeat_scores.csv`, `pooled_predictions.npz` (every candidate's predictions, saved so any later question can be answered without re-running) - `confusion_matrix_review1_ensemble.csv`, `confusion_matrix_review2_best.csv - C_feature_importance.csv`, `C_ranking_agreement.csv`, `C_paper_crosscheck.csv`, `C_feature_subset.csv`, `C_selection_frequency.csv`

Figures: `A_class_balancing.png`, `B1_weight_temperature.png`, `B4_chosen_temperatures.png`, `B_model_comparison.png`, `B_tradeoff.png`, `C1_permutation_importance.png`, `C2_importance_methods_and_direction.png`, `C3_feature_subset.png`.

3.11 `app/` — the web application

`streamlit_app.py` - Top: imports the project's own code (`get_base_learners`, `make_paper_learners`, `load_red_wine_xy`, `FEATURE_COLUMNS`) — so the app uses exactly the project's models. - `MODELS` — the two models' descriptions. - `@st.cache_data load_data()` / `table()` — read the data and result CSVs once, then keep them in memory. - `@st.cache_resource trained_models()` — train **Random Forest** and **RF + SMOTE** on all 1,599 wines **once**, when the app starts; every visitor then reuses them (fast). - Seven page functions: `page_overview` (headline numbers), `page_dataset` (class chart, statistics, correlations), `page_reproduction` (ranking, benchmark, the paper checks), `page_balancing`, `page_improvement`, `page_importance`, `page_predict`. - `page_predict`: a "Start from" box (median of all wines, or of a grade 4–8 wine), 11 sliders, then for each model the predicted grade and a bar chart of probabilities; then **batch prediction**: upload a CSV (comma or semicolon), it checks the 11 columns are present, adds both models' predictions, and offers a download. - Bottom: `st.navigation(pages)` builds the sidebar menu (inside `if __name__ == "__main__"` so the tests can import the pages without starting the app).

`test_app.py` — 15 checks in 4 groups: (1) all 7 pages render without error; (2) the app's two models give **exactly** the same probabilities as freshly trained project models; (3) probabilities sum to 1 and predicted grades are real grades (both models); (4) the shown numbers match the result files (69.01 %, best = RF + SMOTE). Run: `python app/test_app.py` → "ALL CHECKS PASSED".

`requirements.txt` — exact library versions (Streamlit 1.66.0, scikit-learn 1.9.0, imbalanced-learn 0.14.2, …) so the cloud runs exactly what produced the results. **`.streamlit/config.toml`** — the wine-colour theme.

3.12 `notebooks/`

- `wine_quality_colab.ipynb` — a **self-contained** Google Colab notebook: it contains the real code of `src/` and `run_all.py`, downloads the data (or uses a copy built into the notebook if there is no internet). Runs the whole project in a browser.
- `build_colab_notebook.py` — the script that **generates** that notebook from `src/`, so it never goes out of date.
- `reproduction_walkthrough.ipynb` — a step-by-step walkthrough using 2 repeats (writes to a scratch folder).

3.13 `report/` — the report and paper builders

- `build_figures.py` — the report's extra figures (framework diagram, class chart, correlation heat-map, confusion matrices, app architecture, the app-example predictions, output snapshots).
- `build_report.py` + `report_text.py` — build the **69-page Word report** in the SASTRA format; `report_text.py` holds the chapter text; numbers come from `tools/results.py`, references from `tools/refs.py`. `--pages` renders it and fills in the page numbers.
- `build_paper.py` — builds the **conference paper**.
- `tools/docxkit.py` — small helpers for writing Word files (tables, captions, equations via pandoc).
- `tools/make_templates.py` — turned the sample report/paper into **empty** templates (styles and margins only, no content).
- `methodology.md` — the full write-up in plain text.

3.14 `presentation/`

- `build_deck.js` → Review-1 deck; `build_deck_review2.js` → Review-2 deck; **`build_deck_final.js`** → **tomorrow's deck** (`node build_deck_final.js`). The Review-2 and final decks read every number from `results/`.
- `build_review2_docs.py` → wrote the Review-2 briefings and the Review-2 sections of README and methodology.
- `BRIEFING*.md`, `SLIDE_BY_SLIDE*.md`, this guide.

3.15 Small files

- `requirements.txt` — libraries for the whole project. `.gitignore` — what Git must not upload (the paper PDF, caches, quick-run outputs, the decks and finished report/paper).

- `README.md` — project overview. `HOW_TO_RUN.md` — step-by-step commands. `DEPLOY.md` — putting the app online.

Part 4 — Questions the panel may ask

Q1. Why did you get 69 % and not 89.73 %? Is your code wrong? I implemented the paper's equations exactly and tested fairly. Three checks from the paper's own tables point the same way: its confusion matrix has only 491 wines (a single split, not 100 runs) and gives 82.08 %, not 89.73 %; and its own weights times its own model accuracies give about 67.5 %. Our 69.01 % is within 1.5 points of that. Also, the paper's weights are almost equal, which mathematically makes the ensemble an average. I report this as "figures we could not reconcile", not "the paper is wrong", because the table was read from a PDF.

Q2. Couldn't the gap be because you replaced the RNN and the ELM? Unlikely to explain it. Even in the paper, RNN (61.43 %) and ELM (55.77 %) are its weakest models, and with near-equal weights a team cannot beat its best member by 9 points. The paper's own numbers imply ~67.5 % regardless of which models are inside.

Q3. Why did you replace RNN and ELM? They are outside the classical-ML syllabus of the course (RNN = deep learning for sequences; ELM = a network with random, untrained hidden weights). I used the closest classical models: Gradient Boosting (also improves step by step) and an MLP (a neural network trained by back-propagation).

Q4. What is data leakage and how did you avoid it? Leakage is test information sneaking into training — like peeking at exam answers. I put the scaler (and later SMOTE and feature selection) inside a pipeline, so they learn only from the training folds. An early version scaled everything first; that was fixed at Review 1.

Q5. Why 10-fold cross-validation repeated 10 times? To match the paper's "10 runs of 10-fold cross-validation", and because one split is luck-dependent, especially with 10 grade-3 wines. Repeating gives every wine 10 predictions and stable numbers.

Q6. What does "stratified" mean? Each fold has the same mix of grades as the whole data, so every fold contains some rare-grade wines.

Q7. Why is macro-F1 your main measure instead of accuracy? 82.5 % of wines are grade 5 or 6. Always answering "5" already gives 42.6 % accuracy while finding no rare wine. Macro-F1 averages over all six grades equally, so it rewards finding the rare grades — which is what balancing is for. I fixed it in the code before running Review 2 and still report accuracy everywhere.

Q8. What is SMOTE? Why inside the fold? It creates new examples of rare grades by blending a rare wine with one of its 5 nearest rare neighbours. Inside the fold so that no test wine is ever blended into the training data.

Q9. Why did accuracy go down when you balanced? Balancing shifts attention from grades 5/6 to the rare grades. More rare wines are found (grade 4: 0.6 % → 26 %), but some ordinary wines are now called rare. It redistributes correctness rather than creating it — a known, stated trade-off.

Q10. Which is your best model? Depends on the goal. For finding all grades fairly (macro-F1): **Random Forest + SMOTE (40.92)**. For overall accuracy: **Random Forest (71.03 %)**. No model wins both — that is why the app shows both.

Q11. What is the temperature, and isn't choosing it overfitting? T raises the weights to a power so the best model gets more say (T = 1 = paper). The sweep over T is only a diagnostic, because it peeks at the scores. The real result chooses T inside each training fold with an inner 5-fold cross-validation — the test fold never sees the choice — so it is not overfitting to the test data. It improved the paper's own team in 10 of 10 repeats.

Q12. Soft vote vs hard vote? Hard: each model gives one grade; count weighted votes. Soft: add the weighted probabilities. The paper's Eq. 1 is a weighted sum, which reads naturally as soft. I report both (soft 69.01 %, hard 69.93 %); the headline is soft because it is the literal reading, not because it scored higher.

Q13. What is stacking and why did it not help? A second model (logistic regression) learns how to combine the four models' probabilities. Plain stacking reached 70.92 % accuracy but only 34.96 macro-F1 — it learned to favour grades 5 and 6 again. With a balanced combiner it found the most rare wines (balanced accuracy 50 %) but accuracy fell to 59 %.

Q14. Why does Random Forest do so well here? Many varied trees averaging out each other's errors handle noisy tabular data with interacting features well, need no scaling, and are hard to overfit. It is a well-known strong baseline for this kind of table-shaped data.

Q15. Which features matter and why? Alcohol, sulphates and volatile acidity (then chlorides and total SO_2). More alcohol and sulphates → better; more volatile acidity (vinegar taste) → worse. Three different methods agree (0.74–0.89), and 3 of the paper's 4 key attributes are confirmed.

Q16. Why permutation importance and not just Random Forest's built-in importance? The built-in one (MDI) is computed on training data and favours features with many distinct values. Permutation importance is measured on unseen test wines and works for any model, so it shows what the model really relies on. I still show MDI as a cross-check.

Q17. What does a negative importance mean? Scrambling that feature did not hurt (or helped slightly, by chance) — the model gets no usable signal from it, often because another feature carries the same information (density and alcohol are linked).

Q18. How many lab tests are enough? Four (alcohol, volatile acidity, sulphates, chlorides) give 69.27 % — 97 % of the 71.03 % with all eleven, and the best macro-F1. Chosen inside each fold, so no leakage.

Q19. What is the p-value / Wilcoxon test? A test of whether a paired difference across the 10 repeats is consistent or just noise. Small p = consistent. Caveat: the repeats reuse the same wines, so the p-values are optimistic; the win count (e.g. 10/10) is the plainer evidence.

Q20. Grade 3 is found only 4 % of the time even after balancing — why? There are only 10 grade-3 wines in total, about 9 in each training fold. No method can learn a reliable pattern from that few. I state it as a limitation.

Q21. What is your contribution beyond the paper? (1) A leakage-free, repeated-CV reproduction with substitutes; (2) evidence from the paper's own tables for why its number is not reached, and the flat-weights explanation; (3) a tested fix (learned temperature); (4) a fair comparison of balancing methods and 28 candidates under one protocol; (5) feature importance with three cross-checked methods and a "how many tests" analysis; (6) a tested web application, report, paper and fully reproducible code.

Q22. How does the web app work? Streamlit. It imports the project's own model code, trains Random Forest and RF + SMOTE once at start-up (cached), shows the result pages from the result CSVs, and predicts from 11 sliders or a CSV file. 15 automated tests confirm it matches the project.

Q23. Did you tune hyperparameters? Mostly no, on purpose: settings were fixed so comparisons stay fair and to avoid overfitting to the test folds. The only learned setting (the temperature) is chosen inside the training folds.

Q24. Why keep the 240 duplicate rows? The paper uses all 1,599 wines, so for a faithful reproduction we do too. A duplicate can fall into training and test at once, which helps every model equally and does not change the comparison.

Q25. Why classification, not regression? The paper treats grades as six classes, so we reproduce that. Treating grades as ordered (so predicting 6 for a 7 costs less than predicting 3) is future work.

Q26. Could a retailer use this? As a screening aid, yes: early, consistent estimates and a smaller test panel. Not as a replacement for tasters: the rare grades are still hard, and the data has only lab measurements (no price, region or vintage).

Q27. What would you do next? White-wine data, ordinal models, nested cross-validation with proper confidence intervals, and explanations of individual predictions in the app.

Q28. What libraries did you use? Python 3.11; pandas and NumPy (data); scikit-learn (models, pipelines, CV, metrics); imbalanced-learn (SMOTE); SciPy (Wilcoxon, Spearman); Matplotlib (figures); Streamlit and Altair (web app).

Q29. How long does it take to run? Review 1: about 2 minutes (`--quick`: 30 seconds). Review 2: about 45–60 minutes, because it trains thousands of models (28 candidates × 100 fold-runs, plus inner cross-validations).

Q30. Is the ensemble idea useless then? No — the lesson is narrower: an ensemble beats its best member only if the members err differently **and** the weights let the strongest member lead. The paper's normalisation makes the weights too flat; learning the sharpness fixes that.

Part 5 — The numbers on one page

What	Number
Wines / features / grades	1,599 / 11 / 6 (grades 3–8)
Wines per grade (3, 4, 5, 6, 7, 8)	10, 53, 681, 638, 199, 18
Grades 5 + 6 share	82.5 %
Always answering “5”	42.6 % accuracy
Protocol	stratified 10-fold × 10 repeats = 100 fold-runs = 15,990 predictions per model
Paper’s claim	**89.73 %** (+9.02 over its best model, RF 80.71 %)
Paper’s weights	SVM 0.252, RNN 0.229, ELM 0.217, RF 0.303
Our reproduction (soft vote)	**69.01 %** (hard vote 69.93 %)
Our models	RF **71.03**, GB 67.67, SVM 61.95, MLP 61.35 %
Our weights	0.237 / 0.270 / 0.257 / 0.237 (SVM / RF / GB / MLP)
Paper’s Table 9	491 wines (30.7 %), 403 correct = 82.08 % (7.65 below 89.73)
Paper’s weights × paper’s accuracies	67.52 % (ours is +1.49; paper’s claim +22.21)
Review-1 recall per grade	0.0, 0.6, 81.7, 69.9, 49.9, 11.1 %
Balancing: macro-F1 none / R1 / weights / SMOTE	36.64 / 36.92 / 38.77 / 39.34
SMOTE: grade-4 recall, accuracy cost	0.6 → 26.4 %, –5.15 points
Sharpened weights overtake RF at	T = 25 (71.22 % vs 71.03 %)
Learned temperature	**69.01 → 70.89 %**; macro-F1 36.92 → 38.08; T ≥ 10 in 99 % of folds; wins 10/10
Stacking	70.92 % acc but macro-F1 34.96; balanced combiner 59.20 % acc, 50.03 % bal. acc.
Best macro-F1 (28 candidates)	**RF + SMOTE: 40.92** (acc 67.94 %, bal. acc 42.07 %)
Best accuracy	RF: 71.03 %
RF + SMOTE vs Review-1 team	macro-F1 +3.97, 10/10 repeats, p = 0.002
Top features	alcohol, sulphates, volatile acidity (then chlorides, total SO_2)
Method agreement	0.74–0.89
Paper’s 4 key attributes confirmed	3 of 4 (citric acid ranks 6th)
4 tests vs 11	69.27 % vs 71.03 % (97 %)
Web app	7 pages, 2 models, batch CSV, 15/15 checks pass
Report / paper	69 pages, 33 references / 10 pages

Part 6 — Tomorrow

The slide script (short version; full notes are inside the PowerPoint)

Open `final_review.pptx`, press **Presenter View** to see the notes under each slide.

1. **Title** — "Predicting red-wine quality from lab tests; I reproduced a 2024 paper, made three improvements, and built an app."
2. **The question** — lab tests cheap, tasters slow → predict the grade.
3. **Dataset** — 1,599 wines, 11 tests, grades 3–8; 82.5 % are 5 or 6; "always 5" = 42.6 %.
4. **Base paper** — four models, two weights, weighted vote, claims 89.73 %.
5. **Whole project** — the A–D diagram and the three milestones.
6. **Fair testing** — 10 groups, each a turn as test, ×10; no leakage (scaler inside pipeline).
7. **Four models** — SVM, RF kept; GB for RNN, MLP for ELM (syllabus).
8. **Weighted vote** — accuracy weight + diversity weight; both sets of weights ≈ 0.25 → average.
9. **Review-1 result** — 69.01 % vs 89.73 %; RF 71.03 % beats the team; rare grades missed.
10. **Why** — 491 wines; 82.08 %; 67.52 %; caveat about reading the PDF.
11. **The three asks** — map + why macro-F1.
12. **Balancing** — SMOTE best; grade 4: 0.6 → 26.4 %; accuracy –5.
13. **Improving** — temperature; test then fix; 69.01 → 70.89 %; 99 % of folds pick sharp T.
14. **28 candidates** — trade-off; best macro-F1 RF + SMOTE 40.92; best accuracy RF 71.03.
15. **Feature importance** — alcohol, sulphates, volatile acidity; three methods agree; direction.
16. **How many tests** — 4 tests ≈ 97 %.
17. **Before / after** — macro-F1 36.92 → 40.92; accuracy is the price.
18. **Web app** — show it live if possible.
19. **Deliverables** — code, app, report, paper, Colab; one command.
20. **Conclusion** — four lines + the lesson.
21. **Limitations and future work.**
22. **References / thank you.**

Backups (after the thank-you slide): measures in plain words, top candidates, paired comparison, confusion matrices, feature-importance table. Jump to them only if asked.

The demo

If the app is deployed: open the Streamlit link → **Predict a wine** → "Start from: Typical grade-8 wine" → Random Forest says 7, RF + SMOTE says 8 (84 %) → "this is the trade-off in one example".

If there is no internet, run it from the laptop:

```
cd "/Users/AIRUS/Documents/ML Essentials"
source .venv/bin/activate
cd wine-quality-ml-project
streamlit run app/streamlit_app.py
```

It opens at http://localhost:8501. To show the code really runs, in a second terminal: `python run_all.py --quick` (30 seconds; writes to a scratch folder, never overwrites the real results).

Last-minute checklist

- ☐ `final_review.pptx` and `final_review.pdf` copied to a pen drive (the PDF works on any computer)
- ☐ Report and paper PDFs ready in case they ask
- ☐ App link (or laptop with the local app running) — test it once in the morning
- ☐ GitHub page open in a tab: github.com/Surianandhan/wine-quality-ml-project
- ☐ Read Part 5 once more just before you go in

You know this project better than anyone in that room. Speak slowly, point at the numbers, and when you don't know something, say "that would be the next thing to test" — that is an honest and respected answer.